

SIGNALMDM PLATFORM

Enterprise System Documentation Suite

Unified Master Technical Reference & Deployment Specifications

Compilation Date: 2026-05-22 | Version: 1.0.0-RC1

Auditors: Antigravity AI — Principal QA & Platform Architecture Team

Classification: Internal Technical Confidential

System Identifier	SignalMDM (Enterprise Multi-Tenant MDM Platform)
Target Environments	Local Dev / UAT Staging / Production Kubernetes
Core API Layer	FastAPI on Python 3.12 / Uvicorn Engine
Client Interface	React 18 / TypeScript / Vite / Express Gateway
Persistent Tier	PostgreSQL 15+ / Strict Multi-Tenant Schema Scopes
Caching / Security	Redis Queue Broker / AES-256 JWT Decryption Filters

SIGNALMDM: ENTERPRISE MULTI-TENANT MASTER DATA MANAGEMENT PLATFORM

![[License: Proprietary]](LICENSE)

![[Build Status]]()

![[Platform Version]]()

A state-of-the-art, secure, multi-tenant Master Data Management (MDM) platform designed for large-scale enterprise data consolidation, quality assurance, mapping, and auditable data lineage tracking.

1. Project Overview

1.1 Business Purpose

In modern enterprises, data is fragmented across multiple transactional systems, CRM applications, and cloud data warehouses. This fragmentation leads to operational inefficiencies, duplicate records, reporting errors, and compliance risks.

SignalMDM acts as the single source of truth (Golden Record engine) for the enterprise. It ingests records from disparate external source systems, cleanses and normalizes the payload, runs survivorship and mapping rules, and exposes unified data entities. By providing robust logical tenant isolation, administrative audit logs, and granular access control, SignalMDM ensures enterprise data remains consistent, compliant, and highly available.

1.2 System Objectives

- **Single Source of Truth:** Unify multiple representations of the same real-world entity into a single "Golden Record".
- **Strict Multi-Tenancy:** Ensure structural, logical, and transactional isolation between corporate tenants under a single deployment.
- **Security & Compliance First:** Protect sensitive data with client-side encrypted session tokens, device fingerprint verification, sliding-window rate limiting, and immutable database audit logs.
- **Scalable Async Ingestion:** Leverage a robust Celery+Redis task distribution pipeline to load and process millions of records with automated retry capabilities and seamless fallback synchronization.
- **Enterprise Observability:** Track every modification, data drift anomaly, API transaction, and quality exception through intuitive administration dashboard panels.

2. Technology Stack

SignalMDM is engineered using a premium, decoupled architecture separating the API control-plane, background execution threads, caching/throttling layers, and a modern SPA frontend.

2.1 Backend Core

- **FastAPI (v0.136.1):** A high-performance web framework for building APIs, leveraging standard Python type hints.
- **Uvicorn (v0.42.0):** An ultra-fast ASGI web server implementation for launching FastAPI in local and production environments.

- **SQLAlchemy ORM (v2.0.40)**: Unified Object Relational Mapper offering transaction control and dynamic query execution.
- **Pydantic (v2.11.7) & Pydantic-Settings (v2.10.1)**: Strict, runtime typing validation models and secure configuration management.
- **Celery (v5.6.3)**: Distributed asynchronous task execution framework.
- **Redis (v6.2.0)**: High-throughput, low-latency in-memory cache, Celery broker, and sliding rate limiter.

2.2 Security & Authentication

- **PyJWT (python-jose v3.5.0)**: JSON Web Token signing, validation, and claim decoding.
- **Cryptography (v46.0.5)**: AES-256-CBC encryption and decryption of token payloads.
- **Bcrypt (v5.0.0)**: Adaptive hashing for passwords and verification tokens.
- **PyOTP (v2.9.0)**: Time-based and counter-based One-Time Password generation for Multi-Factor Authentication.

2.3 Frontend Application

- **React (v18.3.1)**: A declarative UI component rendering library.
- **TypeScript (v5.5.3)**: Type safety across components, models, and custom React hook integrations.
- **Vite (v5.4.1)**: Ultra-fast compilation, bundling, and hot-module replacement tool.
- **React Router (v6.22.3)**: Dynamic declarative route engine with path matching and guard middleware.
- **Crypto-JS (v4.2.0)**: Client-side payload encryption conforming to AES standards.

2.4 Database & Data Layers

- **PostgreSQL (v15+)**: Advanced open-source object-relational database with strong JSONB support.
- **Psycopg2-binary (v2.9.11)**: High-performance PostgreSQL adapter for Python applications.

3. Major Implemented Features

- **Logic Isolation Engine**: Soft multi-tenancy utilizing foreign key constraints scoped to **tenant_id** tables using a strict **onDelete="RESTRICT"** policy.
- **Encrypted Token Pipeline**: Frontend-encrypted JWT tokens using AES-256-CBC. Decrypted at the backend middleware and validated against SHA-256 device/browser agent fingerprints.
- **Brute-Force Lockout & OTP**: Redis-backed OTP generation delivering secure codes, supported by multi-failed attempt account locks.
- **Multi-Stage Ingestion Pipeline**: Comprehensive state machine (CREATING → RUNNING → RAW_LOADED → STAGING_CREATED → COMPLETED) processing uploads asynchronously via Celery worker chains.
- **Data Quality Pipeline**: High-performance, bulk sanitization layer checking empty values, size boundaries, wildcards, deduping via MD5 hashing, and performing deep regex SQLi/XSS scanning.
- **SuperAdmin Overrides**: Allows members of the **"platform"** tenant to pass **X-Tenant-ID** headers to view and edit distinct tenant scopes dynamically.
- **Immutable Append-Only Audit**: Persistent history log preserving JSONB old/new snapshots and correlating transactions via trace IDs.
- **Sliding-Window Rate Limiter**: Throttles high-risk endpoints (**/auth/login**) backed by Redis.

4. Repository Structure

```
SignalMDM/  
■■■ MDM_Backend/ # FastAPI Backend API Server  
■ ■■■ core/ # Centralized settings & Redis connection pool
```

```
■ ■■■ scripts/ # SQL migration & platform seeding scripts
■ ■■■ signalmdm/ # Core Application Package
■ ■ ■■■ models/ # SQLAlchemy ORM schemas
■ ■ ■■■ schemas/ # Pydantic v2 schemas
■ ■ ■■■ services/ # Core business logic handlers
■ ■ ■■■ routers/ # API route controllers
■ ■ ■■■ middleware/ # Decryption, Auth guards, & Rate-limiting
■ ■ ■■■ workers/ # Celery background tasks
■ ■■■ storage/ # Ingested file buffer storage
■ ■■■ utils/ # Hash generators and path sanitizers
■ ■■■ main.py # FastAPI Application Entrypoint
■ ■■■ signalmdm_security_tester.py # Dynamic pentest suite
■
■■■■ MDM_DataLayer/ # Database Artifacts
■ ■■■ SignalMDM.sql # Full PostgreSQL schema binary dump
■
■■■■ MDM_Frontend/ # React SPA Client App
■ ■■■ src/ # Source Code
■ ■ ■■■ components/ # Reusable UI controls and tables
■ ■ ■■■ context/ # Contexts (Auth, Tenant, Theme)
■ ■ ■■■ layouts/ # Shells (Dashboard sidebars)
■ ■ ■■■ pages/ # Views (Ingestion, Tenants, RBAC)
■ ■ ■■■ services/ # API client services
■ ■ ■■■ utils/ # Crypto wrappers and helpers
■ ■■■ serverClient.js # Production Express Server with Helmet CSP
■ ■■■ vite.config.ts # Vite asset bundler config
■■■■ README.md # Home documentation page
```

5. Quick Start & Local Development

This section covers full local configuration of SignalMDM on a development machine.

5.1 Environment Prerequisites

Ensure the following tools are installed locally:

- **Python:** v3.10+ (Recommended: v3.12)
- **Node.js:** v18+ (Recommended: v20 LTS)
- **PostgreSQL:** v15+ (Local port **5432** with username **postgres**)
- **Redis:** v7+ (Or **Memurai** for Windows installations)

5.2 Step 1: Database Setup

1. Create the Database:

Connect to PostgreSQL using pgAdmin or psql and run:

```
CREATE DATABASE "SignalMDM" WITH ENCODING = 'UTF8';
```

2. Restore the Schema:

The base schema dump is in **MDM_DataLayer/SignalMDM.sql**. Open a shell and run:

```
psql -U postgres -d SignalMDM -f "d:\SignalMDM\MDM_DataLayer\SignalMDM.sql"
```

3. Seed Platform Administrators & RBAC Matrix:

You must execute the setup and RBAC scripts to enable administrative authentication:

```
psql -U postgres -d SignalMDM -f "d:\SignalMDM\MDM_Backend\scripts\platform_admin_setup.sql"
psql -U postgres -d SignalMDM -f "d:\SignalMDM\MDM_Backend\scripts\platform_rbac_migration.sql"
```

- **Seed Account Email:** `admin@signalmdm.com`
- **Seed Account Password:** `Admin@Signal123`
- **Assigned Seed Role:** `super_admin`

5.3 Step 2: Caching & Task Broker (Redis)

Start the Redis server on your local system:

- **Windows (Memurai):**

```
net start Memurai
```

- **Linux/macOS:**

```
redis-server
```

Confirm Redis is listening:

```
redis-cli ping
# Expected response: PONG
```

5.4 Step 3: Backend Configuration (MDM_Backend/)

1. Initialize Virtual Environment:

```
cd d:\SignalMDM\MDM_Backend
python -m venv venv
```

- *Activate (Windows PowerShell):* `.\venv\Scripts\Activate.ps1`
- *Activate (Linux/macOS):* `source venv/bin/activate`

2. Install Python Dependencies:

```
pip install fastapi uvicorn sqlalchemy psycpg2-binary pydantic pydantic-settings python-dotenv celery
redis python-multipart "python-jose[cryptography]" cryptography bcrypt pyotp email-validator
```

3. Create local `.env` file:

Verify and configure `MDM_Backend/.env`:

```
DATABASE_URL=postgresql://postgres:2025@localhost:5432/SignalMDM
REDIS_URL=redis://localhost:6379/0
JWT_SECRET=supersecretkey
JWT_ALGORITHM=HS256
JWT_EXPIRE_MINUTES=1440
TOKEN_ENCRYPTION_KEY=a3f1b2c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2
APP_ENV=development
UPLOAD_DIR=storage/uploads
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASSWORD=your_app_password
SMTP_FROM=no-reply@signalmdm.com
SMTP_USE_TLS=True
```

4. Create Upload Buffer:

```
mkdir -p storage/uploads
```

5. Verify Routing Integrity:

```
python -c "import main; print('OK:', len(main.app.routes), 'routes compiled')"  
# Expected: OK: 17 routes compiled
```

5.5 Step 4: Run Backend Applications

Start two separate command-line shells to run the API and task queue workers:

- **Terminal A: FastAPI Web Server**

```
cd d:\SignalMDM\MDM_Backend  
# Ensure venv is active  
python -m uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

API Docs are visible at: <http://localhost:8000/docs>

- **Terminal B: Celery Worker Task Processor**

```
cd d:\SignalMDM\MDM_Backend  
# Ensure venv is active  
# Solo pool is mandatory for Windows environments  
python -m celery -A signalmdm.workers.celery_app worker --loglevel=info --pool=solo
```

5.6 Step 5: Frontend Configuration (MDM_Frontend/)

1. Install Node Modules:

```
cd d:\SignalMDM\MDM_Frontend  
npm install
```

2. Verify Frontend Settings (.env.local):

Create **MDM_Frontend/.env.local**:

```
VITE_API_BASE_URL=http://localhost:8000/api/v1  
VITE_TOKEN_ENCRYPTION_KEY=a3f1b2c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2
```

3. Run Development Mode (Hot-Reloading):

```
npm run dev
```

Frontend is available at <http://localhost:5173>.

4. Compile & Run Production Mode (Strict CSP Testing):

To validate headers and Helmet CSP compliance, compile and run the production build using the Express server wrapper:

```
npm run build  
npm run serve
```

This serves compiled static assets with strict Content-Security-Policies on <http://localhost:3030>.

6. Testing

Dynamic security verification and API scanning can be validated using the built-in pentester tool:

1. Ensure the FastAPI server is running on <http://localhost:8000>.
2. Open a shell inside **MDM_Backend/** and execute:

```
python signalmdm_security_tester.py
```

This triggers **115 active test scenarios** checking JWT parameters, brute force lockouts, SQL injection, XSS reflection, and traversal attacks.

7. Links to System Documentation

For detailed guides, please refer to the specialized documentation files:

- **ARCHITECTURE.md**: System architecture, Logical layouts, Request lifecycles, and complete Mermaid flows.
- **DIRECTORY_REFERENCE.md**: File-by-file explanations of classes, methods, and functions.
- **API_REFERENCE.md**: Endpoint list, authorization parameters, validation checks, and payloads.
- **DATABASE_REFERENCE.md**: PostgreSQL schema catalogs, multi-tenant restrictions, and audit structures.
- **DEPLOYMENT.md**: Kubernetes deployment matrices, Dockerfiles, and incident runbooks.
- **TECHNICAL_REFERENCE.md**: Deep-dive reference guides for third-party libraries.

8. Known Limitations & Roadmap

8.1 Current Limitations

- **Permissive CSP in Dev**: The Vite HMR model requires **'unsafe-inline'** and **'unsafe-eval'** policies inside Helmet, which are disabled only during standard client-build compilations.
- **No Centralized Storage abstraction**: File chunks are stored directly in the local file system. Production setups require a custom S3/Cloud Storage object driver.
- **Mock Dashboard Telemetry**: The main analytical charts on the layout interface populate simulated telemetry counts, with real data hook integration scheduled for Phase 2.

8.2 Roadmap

1. **Phase 2 Entity Resolution**: Integration of probabilistic matching models, Jaro-Winkler/Levenshtein algorithms, and graph matching schemas.
2. **Machine Learning Scoring Engine**: Integrate advanced scoring modules (**scoring.py**, **features.py**) to run anomaly detection models on raw inputs.
3. **Real-Time Distributed Ingestion**: Deploy Apache Kafka event ingestion queues to stream live signal changes.

SIGNALMDM: SYSTEMS & ARCHITECTURAL SPECIFICATION

This document provides a comprehensive technical overview of the system architecture, logical layers, request pipelines, and background execution topologies of the SignalMDM platform.

1. System Topology & Logical Architecture

SignalMDM is designed around a decoupled, three-tier enterprise service architecture. The control plane, storage interfaces, task-queues, and caching subsystems operate independently, ensuring horizontal scale and structural high availability.

1.1 Architectural Component Topology

```
graph TD
    subgraph ClientLayer ["Client Layer (SPA)"]
        Browser["Web Browser (React SPA)"]
        CryptoJS["CryptoJS (Client Encrypter)"]
        Browser --> CryptoJS
    end

    subgraph ReverseProxy ["Reverse Proxy & Web Server"]
        Express["Express SPA Host (port: 3030)"]
        Helmet["Helmet Middleware (CSP/Headers)"]
        Express --> Helmet
    end

    subgraph ControlPlane ["Control Plane (API Server)"]
        FastAPI["FastAPI Engine (port: 8000)"]
        Uvicorn["Uvicorn Server Process"]
        RateLimit["Rate Limit Middleware"]
        DecryptMW["AES Decrypt Middleware"]
        AuthMW["Auth Guard Middleware"]
        FastAPI --> Uvicorn
        Uvicorn --> RateLimit
        RateLimit --> DecryptMW
        DecryptMW --> AuthMW
    end

    subgraph DataCache ["State & Cache Layer"]
        Redis["Redis Memory Cache (port: 6379)"]
        Blacklist["Token Revocation Hash"]
        OTPStore["OTP Token Hash"]
        Redis --> Blacklist
        Redis --> OTPStore
    end

    subgraph BackgroundWorkers ["Async Processing Tier"]
        Celery["Celery Task Manager"]
        RawWorker["Raw Extractor Worker"]
        StagingWorker["Staging Processor Worker"]
        Celery --> RawWorker
        RawWorker --> StagingWorker
    end

    subgraph StorageTier ["Data Storage Tier"]
```

```

    Postgres[("PostgreSQL Database (port: 5432)")]
    DiskStorage["Local File Store (/storage)"]
end

Browser -->|Encrypted Session / Static Web Request| Express
Express -->|API Ingestion Request| Uvicorn
AuthMW -->|Verify Revocation| Redis
Celery -->|Queue Broker Tasks| Redis
FastAPI -->|Async Job Delegation| Celery
RawWorker -->|Load File Chunks| DiskStorage
RawWorker -->|Insert Raw JSONB| Postgres
StagingWorker -->|Transform and Align| Postgres
FastAPI -->|Direct DB Operations / Sync Fallback| Postgres

```

2. Request Lifecycle & Authentication Flows

Every API request entering the SignalMDM boundary is intercepted, audited, decrypted, and evaluated before reaching the route business logic.

2.1 The Request Lifecycle Flow

```

sequenceDiagram
    autonumber
    participant Browser as Web Browser (React SPA)
    participant Express as Express SPA (Port 3030)
    participant RateLimit as Redis Rate Limiter
    participant Decrypt as AES Decryption Middleware
    participant Auth as Auth Middleware Guard
    participant Route as FastAPI Router Endpoint
    participant DB as PostgreSQL DB Engine

    Browser->>Express: HTTPS GET /api/v1/sources
    Note over Browser, Express: Payload contains VITE_TOKEN_ENCRYPTION_KEY encrypted JWT

    Express->>RateLimit: Request through rate limit check
    alt Rate limit exceeded (e.g. >20 req/min)
        RateLimit-->>Browser: HTTP 429 Too Many Requests
    else Limit OK
        RateLimit->>Decrypt: Forward payload
    end

    Decrypt->>Decrypt: Extract AES cipher in header/cookie
    alt Decryption fails (missing/malformed)
        Decrypt-->>Browser: HTTP 400 Bad Request
    else Decryption Successful (Plain JWT output)
        Decrypt->>Auth: Hand off decoded credentials
    end

    Auth->>Auth: Validate JWT Signature (HS256 Only)
    Auth->>Auth: Verify expiry time & device fingerprint (SHA256)
    alt Validation fails / Expired
        Auth-->>Browser: HTTP 401 Unauthorized
    else Validation Successful
        Auth->>Route: Mount user context in route dependencies
    end

    Route->>DB: Perform isolated database query (X-Tenant-ID scoping)

```

```
DB-->>Route: Return query dataset
Route-->>Browser: HTTP 200 OK (Cleaned JSON payload)
```

2.2 Secure Encrypted Authentication Flow

The security pipeline utilizes client-side encryption to prevent JWT inspection or token interception, combined with multi-factor OTP validation and automated rate limiting.

```
graph TD
    A["User submits username/password (Login.tsx)"] --> B["API checks credentials (bcrypt)"]
    B -->|Success| C["Generate dynamic 6-digit OTP code"]
    C --> D["Write OTP salt and hash to Redis (10 min TTL)"]
    D --> E["Transmit OTP code to User Email (SMTP)"]
    E --> F["User submits OTP code"]
    F --> G["API verifies OTP in Redis"]
    G -->|Valid| H["Generate JSON Web Token (JWT)"]
    H --> I["Encrypt JWT with AES-256-CBC (VITE_TOKEN_ENCRYPTION_KEY)"]
    I --> J["Set secure HTTPOnly cookie (accessToken)"]
```

3. Data Ingestion & Async Processing Flow

The SignalMDM ingestion engine is an asynchronous state machine managed by Celery. It coordinates file landing-zone persistence, schema integrity validation, deduplication, and staging translation.

3.1 Upload Processing Flow

```
graph TD
    A["User initiates file upload (UploadData.tsx)"] -->|File stream| B["upload_router.py (POST /upload)"]
    B --> C["Sanitize path with os.path.basename()"]
    C --> D["Generate unique session upload run_id"]
    D --> E["Stream file chunks to local disk /storage/uploads/{run_id}"]
    E --> F["Generate final MD5 checksum of file"]
    F --> G["Write upload metadata to PostgreSQL (UploadSession)"]
    G --> H["Hand off execution to background task queue"]
```

3.2 Ingestion & Queue Processing Flow

Once a file is safely landed, the FastAPI control-plane dispatches tasks to the Celery broker (Redis), orchestrating an asynchronous processing pipeline.

```
sequenceDiagram
    autonumber
    participant Router as API Ingestion Router
    participant Queue as Redis Queue Broker
    participant Raw as Celery: Raw Ingest Worker
    participant Staging as Celery: Staging Ingest Worker
    participant DB as PostgreSQL Database

    Router->>DB: Set IngestionRun status = RUNNING
    Router->>Queue: Dispatch task: raw_worker.process_file(run_id)
    Router-->>Router: HTTP 202 Accepted (Returns run_id to Client)

    activate Raw
    Queue->>Raw: Consume raw ingestion job
    Raw->>DB: Load file metadata & open data stream
```

```

Raw->>Raw: Execute Data Quality Pipeline (sanitize.py)<br/>- Dedup rows via MD5 hash<br/>- Regex scan
columns for SQLi/XSS payloads<br/>- Filter oversized fields
Raw->>DB: Bulk insert records into platform.raw_records table
Raw->>DB: Update IngestionRun status = RAW_LOADED
Raw->>Queue: Dispatch task: staging_worker.process_run(run_id)
deactivate Raw

activate Staging
Queue->>Staging: Consume staging task
Staging->>DB: Map 1-to-1 raw_records to staging_entities
Staging->>DB: Verify target schemas & correlation IDs
Staging->>DB: Apply matching/survivorship guidelines
Staging->>DB: Update IngestionRun status = COMPLETED
deactivate Staging

```

4. Multi-Tenant Isolation Architecture

Logical multi-tenancy is enforced natively in the database schema.

4.1 Structural Database Relationship Design

Every entity, source system, mapping config, and transaction log is bound to the root organisation via the **tenant_id** foreign key.

```

erDiagram
    TENANT ||--o{ APP_USER : "has members"
    TENANT ||--o{ SOURCE_SYSTEM : "defines adapters"
    TENANT ||--o{ AUDIT_LOG : "generates history"
    TENANT ||--o{ INGESTION_RUN : "executes runs"
    TENANT ||--o{ RAW_RECORD : "ingests"
    TENANT ||--o{ STAGING_ENTITY : "holds staging"

    TENANT {
        uuid tenant_id PK
        string name
        jsonb config_json
    }
    APP_USER {
        uuid user_id PK
        uuid tenant_id FK
        string email
        string password_hash
    }
    SOURCE_SYSTEM {
        uuid source_id PK
        uuid tenant_id FK
        string code
        string name
    }
    INGESTION_RUN {
        uuid run_id PK
        uuid tenant_id FK
        string status
    }
    RAW_RECORD {
        uuid raw_record_id PK
        uuid tenant_id FK
    }

```

```

    jsonb raw_data
    string checksum_md5
  }
  STAGING_ENTITY {
    uuid staging_id PK
    uuid tenant_id FK
    uuid raw_record_id FK
    string state
  }
}
```

4.2 Query-Level Tenant Scoping Pipeline

1. Direct Routing Scope:

Standard tenants ([admin](#), [data_architect](#), [data_manager](#)) can only query records matching their session context [tenant_id](#) (extracted from the authenticated JWT payload). The database query layers automatically append **WHERE tenant_id = :session_tenant_id** to all SELECT, UPDATE, and DELETE operations.

2. SuperAdmin Scoping Overrides:

SuperAdmins belonging to the **"platform"** tenant can override the target tenant context:

- The browser app sends a specific header **X-Tenant-ID** containing the target UUID.
- The [auth.py](#) middleware parses this header, validates that the user is a [super_admin](#), and mounts the requested **X-Tenant-ID** as the active [tenant_id](#) parameter inside the request database session context.

3. Referential Integrity Safeguard:

All database tables declare:

```
tenant_id = Column(UUID(as_uuid=True), ForeignKey("tenant.tenant_id", ondelete="RESTRICT"), nullable=False)
```

This constraint prevents accidental deletions or cascades from wiping out organizational master data.

5. Deployment Topology

SignalMDM is deployed as a resilient micro-services cluster managed by Kubernetes.

```

graph TD
    subgraph Internet ["Public Network"]
        Client["Web Browser"]
    end

    subgraph DMZ ["Kubernetes Cluster Boundary"]
        Ingress["Nginx Ingress Controller (TLS Termination)"]
        WebPod["Frontend Service Pod (Express)"]
    end

    subgraph PrivateSubnet ["Isolated Backend Services"]
        APIPod["Backend API Service Pods (FastAPI)"]
        WorkerPod["Celery Task Processing Pods"]
        RedisCluster["Redis Replication Set (Rate Limiting/Queue)"]
    end

    subgraph DataStorage ["Secure Databases Layer"]
        PostgresReplica["PostgreSQL Master/Replica Set"]
    end

    Client -->|HTTPS / Port 443| Ingress
```

```
Ingress -->|Static Bundle / SPA Assets| WebPod
Ingress -->|Forward API Traffic /api/v1| APIPod
APIPod -->|Broker Jobs| RedisCluster
WorkerPod -->|Consume Queued Jobs| RedisCluster
APIPod -->|Query / Mutate| PostgresReplica
WorkerPod -->|Query / Mutate| PostgresReplica
```

SIGNALMDM: DIRECTORY REFERENCE & MODULE CATALOG

This document provides a highly detailed structural reference mapping every directory and critical file in the SignalMDM enterprise repository.

1. Directory Structure Map

```
SignalMDM/
├── MDM_Backend/ # Core FastAPI control-plane & async execution engine
│   ├── core/ # Configuration schemas and cache connection setups
│   ├── scripts/ # SQL seed and initial database migrations
│   ├── signalmdm/ # Python backend package
│   ├── middleware/ # Encryption filters, auth guards, rate limit checkers
│   ├── models/ # SQLAlchemy transactional definitions
│   ├── routers/ # FastAPI route controllers
│   ├── schemas/ # Pydantic v2 data transfer objects (DTO)
│   ├── services/ # Central transactional business handlers
│   ├── workers/ # Celery worker process entries
│   ├── storage/ # Local buffer for file ingestion operations
│   ├── utils/ # Data verification, paths, and sanitize libraries
│   └──
├── MDM_DataLayer/ # DB restoration files and structure artifacts
└── MDM_Frontend/ # React SPA user interface
    ├── src/ # Source workspace
    │   ├── components/ # Tables, layouts, indicators, dynamic boxes
    │   ├── context/ # React central context hooks (Auth, Snackbar, Tenant)
    │   ├── layouts/ # Dashboard structural shells
    │   ├── pages/ # Main React view panels
    │   ├── services/ # Axios API integration drivers
    │   └── utils/ # CryptoJS helpers and fingerprint engines
```

2. Directory Reference Details

2.1 Backend Directories

MDM_Backend/core/

- **Purpose:** Application core setups, variables parsing, database credentials pooling.
- **Responsibilities:** Loads Pydantic models from `.env`, creates and maintains single-instance Redis pools.
- **Contained Modules:** `config.py`, `redis_client.py`.
- **Dependencies:** `pydantic-settings`, `redis`.
- **Used By:** Routers, Services, Middlewares.
- **Examples:** Initializing Redis connection pools on application startup (`main.py`).

MDM_Backend/signalmdm/middleware/

- **Purpose:** Intercepts incoming requests for authentication and sanitization.
- **Responsibilities:** Rate limiting, JWT decryption, user fingerprinting, injection scanning.
- **Contained Modules:** `auth.py`, `token_utils.py`, `rate_limit.py`.

- **Dependencies:** `cryptography`, `jose`, `redis`, `fastapi`.
- **Used By:** `main.py` routing lifecycle.

MDM_Backend/signalmdm/models/

- **Purpose:** PostgreSQL database representation tables.
- **Responsibilities:** Represents core data layers, joins, and integrity constraints.
- **Contained Modules:** `tenant.py`, `platform_admin.py`, `platform_role.py`, `rbac.py`, `source_system.py`, `file_upload.py`, `upload_session.py`, `ingestion_run.py`, `raw_record.py`, `staging_entity.py`, `entity.py`, `audit.py`, `scoring.py`.
- **Dependencies:** `sqlalchemy`.
- **Used By:** Services, Routers.

MDM_Backend/signalmdm/services/

- **Purpose:** Encapsulates the core transactional and logic rules of the system.
- **Responsibilities:** Drives token workflows, processes source operations, maps raw databases.
- **Contained Modules:** `auth_service.py`, `source_service.py`, `ingestion_service.py`, `raw_service.py`, `staging_service.py`, `audit_service.py`, `tenant_service.py`.
- **Dependencies:** `sqlalchemy`, `bcrypt`, `pyotp`.
- **Used By:** Routers, Workers.

MDM_Backend/signalmdm/routers/

- **Purpose:** API Controllers exposing HTTP endpoints.
- **Responsibilities:** Maps endpoints, resolves path parameters, delegates workflows to services.
- **Contained Modules:** `auth_router.py`, `source_router.py`, `ingestion_router.py`, `raw_router.py`, `staging_router.py`, `upload_router.py`, `tenant_router.py`, `platform_rbac_router.py`.
- **Dependencies:** `fastapi`, `sqlalchemy`.
- **Used By:** Mounted in `main.py`.

2.2 Frontend Directories

MDM_Frontend/src/context/

- **Purpose:** Global reactive state management contexts.
- **Responsibilities:** Maintains logins, updates active tenants, triggers snackbar alerts.
- **Contained Modules:** `AuthContext.tsx`, `PermissionsContext.tsx`, `TenantConfigContext.tsx`, `SnackbarContext.tsx`.
- **Dependencies:** `react`, `js-cookie`.
- **Used By:** Standard React Components.

MDM_Frontend/src/pages/

- **Purpose:** Contains UI view components.
 - **Responsibilities:** Renders forms, mounts tables, handles UI bindings.
 - **Contained Modules:** `Login.tsx`, `SourceSystems.tsx`, `IngestionRuns.tsx`, `UploadData.tsx`, `RawLanding.tsx`, `StagingRecords.tsx`, `PlatformRBAC.tsx`.
 - **Dependencies:** `react`, `lucide-react`, `crypto-js`.
 - **Used By:** Mounted in `App.tsx` routes.
-

3. Module Catalog: File-by-File Breakdown

3.1 Backend Module Details

MDM_Backend/main.py

- **Purpose:** Main application launcher and middleware coordinator.
- **Major Classes:** None.
- **Major Functions:** `startup_event()`, `shutdown_event()`, `custom_validation_exception_handler()`.
- **Dependencies:** `fastapi`, `uvicorn`, `redis_client`, `auth_middleware`.
- **Used By:** System execution runtime.
- **Business Role:** Binds the FastAPI routing layer, coordinates startup health checks, and mounts middlewares.

MDM_Backend/signalmdm/middleware/auth.py

- **Purpose:** Enforces token verification and security guards.
- **Major Classes:** `SecurityHeadersMiddleware`, `ResponseEnvelopeMiddleware`.
- **Major Functions:** `require_auth()`, `require_admin()`, `is_super_admin()`.
- **Dependencies:** `fastapi`, `token_utils`, `redis_client`, `sqlalchemy`.
- **Used By:** Routers, Controllers.
- **Business Role:** Validates encrypted JWT cookie assertions, checks device IDs, and maps role permissions.

MDM_Backend/signalmdm/middleware/token_utils.py

- **Purpose:** Provides token cryptographic signature and envelope operations.
- **Major Classes:** None.
- **Major Functions:** `aes_encrypt()`, `aes_decrypt()`, `create_access_token()`, `decode_token()`.
- **Dependencies:** `cryptography`, `jose`, `core.config`.
- **Used By:** `auth.py`, `auth_service.py`.
- **Business Role:** Restricts token payload viewing via AES-256 wrapping and validates JWT claims.

MDM_Backend/signalmdm/middleware/rate_limit.py

- **Purpose:** Redis-backed request throttling middleware.
- **Major Classes:** `RateLimitingMiddleware`.
- **Major Functions:** `dispatch()`.
- **Dependencies:** `fastapi`, `redis_client`, `core.config`.
- **Used By:** Mounted in `main.py`.
- **Business Role:** Implements sliding rate-limiting limits to prevent brute-force attacks.

MDM_Backend/signalmdm/services/auth_service.py

- **Purpose:** Core identity controller for the platform.
- **Major Classes:** `AuthService`.
- **Major Functions:** `authenticate_admin()`, `verify_otp()`, `verify_2fa()`, `refresh_session()`.
- **Dependencies:** `sqlalchemy`, `bcrypt`, `pyotp`, `redis_client`, `token_utils`.
- **Used By:** `auth_router.py`.
- **Business Role:** Processes hashes, generates temporary OTP secrets, and handles session caching.

MDM_Backend/signalmdm/services/ingestion_service.py

- **Purpose:** Processes file loading workflows and state machines.
- **Major Classes:** `IngestionService`.
- **Major Functions:** `create_run()`, `transition_status()`, `process_run_sync()`.
- **Dependencies:** `sqlalchemy`, `raw_service`, `staging_service`.
- **Used By:** `ingestion_router.py`, workers.
- **Business Role:** Orchestrates ingestion state boundaries, handling fail-open sync operations.

MDM_Backend/signalmdm/utils/sanitize.py

- **Purpose:** Comprehensive Data Quality pipeline processor.
- **Major Classes:** None.
- **Major Functions:** `clean_payload()`, `scan_injection()`, `deduplicate_batch()`.
- **Dependencies:** `re`, `json`, `hashlib`.
- **Used By:** `raw_worker.py`, services.
- **Business Role:** Audits incoming datasets to filter anomalies, detect injection attempts, and ensure clean ingestion.

3.2 Frontend Module Details

MDM_Frontend/src/context/AuthContext.tsx

- **Purpose:** Manages global client login state.
- **Major Classes:** `AuthProvider`.
- **Major Functions:** `login()`, `logout()`, `verifyOtp()`.
- **Dependencies:** `react`, `js-cookie`, `crypto-js`.
- **Used By:** Header dashboards, protected views.
- **Business Role:** Manages client-side login sessions and session storage caches.

MDM_Frontend/src/pages/IngestionRuns.tsx

- **Purpose:** Displays ingestion workflows and system health stats.
- **Major Classes:** None.
- **Major Functions:** `IngestionRuns()`.
- **Dependencies:** `react`, `lucide-react`, `TenantConfigContext`.
- **Used By:** Shell routing.
- **Business Role:** Provides administrators with real-time tracking of data loads.

MDM_Frontend/src/pages/PlatformRBAC.tsx

- **Purpose:** Dynamic user roles configuration matrix.
- **Major Classes:** None.
- **Major Functions:** `PlatformRBAC()`.
- **Dependencies:** `react`, `PermissionsContext`.
- **Used By:** Shell routing.
- **Business Role:** Allows super-admins to modify role scopes and system permissions.

SIGNALMDM: API REFERENCE MANUAL

This document details all API endpoints exposed under the SignalMDM application control-plane `/api/v1` namespace.

1. Global API Parameters

1.1 Request Headers

Every authenticated endpoint requires the following headers:

Header	Format	Description
Authorization	Bearer <AES-256-encrypted-JWT>	Base session token, encrypted using the global <code>TOKEN_ENCRYPTION_KEY</code> .
X-Device-ID	UUIDv4	Unique device identifier used to compile the SHA-256 fingerprint constraint.
User-Agent	String	Raw browser details matched against the fingerprint during verification.
X-Tenant-ID	UUIDv4	(Optional) Used by Platform SuperAdmins to override the current tenant query boundary.

2. Endpoint Catalog

2.1 Authentication Control Layer (/api/v1/auth)

POST /api/v1/auth/login

- **Purpose:** Initial phase authentication. Validates username and password, then triggers OTP code delivery.
- **Authentication:** None.
- **Implementation Location:** `MDM_Backend/signalmdm/routers/auth_router.py` (L45)
- **Handler:** `login()`
- **Service:** `AuthService.authenticate_admin()`
- **Request Schema:**

```
{
  "email": "user@tenant.com",
  "password": "Password123"
}
```

- **Response Schema (200 OK):**

```
{
  "message": "OTP delivered. Validate verification session.",
  "admin_id": "a94037cf-e989-4823-b066-e83b708602c8",
  "mfa_required": true
}
```

- **Error Responses:**

- **401 Unauthorized:** "Invalid credentials."
- **429 Too Many Requests:** Rate limit triggered (Max 5 attempts/minute).
- **Business Rules:** Returns identical "Invalid credentials" messages for both incorrect passwords and non-existent accounts to prevent user enumeration.

POST /api/v1/auth/verify-otp

- **Purpose:** Validates the temporary 6-digit OTP code and issues the encrypted JWT access token.
- **Authentication:** None.
- **Implementation Location:** `MDM_Backend/signalmdm/routers/auth_router.py` (L82)
- **Handler:** `verify_otp()`
- **Service:** `AuthService.verify_otp()`
- **Request Schema:**

```
{
  "admin_id": "a94037cf-e989-4823-b066-e83b708602c8",
  "otp_code": "123456"
}
```

- **Response Schema (200 OK):**

```
{
  "access_token": "U2FsdGVkX19xV...",
  "token_type": "bearer"
}
```

- **Error Responses:**
 - **401 Unauthorized:** "OTP token expired or invalid."
 - **403 Forbidden:** "Account locked. Maximum OTP validation attempts exceeded."
- **Business Rules:** OTP validation limits attempts to 5 before locking out the admin ID.

GET /api/v1/auth/me

- **Purpose:** Returns profile details for the currently active authenticated session.
- **Authentication:** Required.
- **Permissions:** Any valid active role.
- **Implementation Location:** `MDM_Backend/signalmdm/routers/auth_router.py` (L120)
- **Handler:** `get_me()`
- **Response Schema (200 OK):**

```
{
  "admin_id": "a94037cf-e989-4823-b066-e83b708602c8",
  "email": "admin@signalmdm.com",
  "full_name": "Root Administrator",
  "role": "super_admin",
  "tenant_id": "platform"
}
```

2.2 Source Systems Management (/api/v1/sources)

POST /api/v1/sources/register

- **Purpose:** Registers a new external source system integration.

- **Authentication:** Required.
- **Permissions:** `admin`, `super_admin`, `data_architect`.
- **Implementation Location:** `MDM_Backend/signalmdm/routers/source_router.py` (L35)
- **Handler:** `register_source()`
- **Service:** `SourceService.register_source()`
- **Request Schema:**

```
{
  "code": "CRM-CORP",
  "name": "Corporate CRM Adapter",
  "description": "Consolidates corporate CRM contact profiles.",
  "config_json": {
    "sync_frequency": "hourly",
    "api_schema": {
      "mapping_id": "9cbae21f-1218-4825-9c9e-ca8cd0857030"
    }
  }
}
```

- **Response Schema (201 Created):**

```
{
  "source_id": "7b093dcf-18e4-4d81-8b01-a67b9393e9a1",
  "code": "CRM-CORP",
  "status": "active"
}
```

- **Validation Rules:** Code must be uppercase, alphanumeric, and between 3-10 characters.

2.3 File Upload & Ingestion (/api/v1/upload & /api/v1/ingestion)

POST /api/v1/upload

- **Purpose:** Uploads raw multi-part CSV/JSON files for processing.
- **Authentication:** Required.
- **Permissions:** `admin`, `data_architect`, `data_manager`.
- **Implementation Location:** `MDM_Backend/signalmdm/routers/upload_router.py` (L28)
- **Handler:** `upload_file()`
- **Response Schema (200 OK):**

```
{
  "session_id": "f5b61c92-34ab-4673-89bd-2b00567ef890",
  "filename": "contacts_2026.csv",
  "size_bytes": 1048576,
  "checksum_md5": "9e107d9d372bb6826bd81d3542a419d6"
}
```

- **Business Rules:** Filenames are automatically sanitized using `os.path.basename()` to prevent path traversal attacks.

POST /api/v1/ingestion/run

- **Purpose:** Triggers the asynchronous parsing and staging task chain.
- **Authentication:** Required.
- **Permissions:** `admin`, `data_architect`, `data_manager`.

- **Implementation Location:** `MDM_Backend/signalmdm/routers/ingestion_router.py` (L45)
- **Handler:** `trigger_ingestion()`
- **Service:** `IngestionService.create_run()`
- **Request Schema:**

```
{
  "source_id": "7b093dcf-18e4-4d81-8b01-a67b9393e9a1",
  "session_id": "f5b61c92-34ab-4673-89bd-2b00567ef890"
}
```

- **Response Schema (202 Accepted):**

```
{
  "run_id": "3cbae21f-1218-4825-9c9e-ca8cd0857030",
  "status": "RUNNING",
  "message": "Async ingestion task successfully queued."
}
```

- **Business Rules:** Automatically falls back to synchronous processing if the Celery worker queue is unavailable, returning a **200 OK** once processing completes.

2.4 Platform Role-Based Access Control (`/api/v1/platform-rbac`)

GET `/api/v1/platform-rbac/my-permissions`

- **Purpose:** Fetches the current user's navigation permission matrix.
- **Authentication:** Required.
- **Permissions:** Any valid session.
- **Implementation Location:** `MDM_Backend/signalmdm/routers/platform_rbac_router.py` (L25)
- **Response Schema (200 OK):**

```
{
  "permissions": [
    {
      "permission_id": "9cbae21f-1218-4825-9c9e-ca8cd0857030",
      "screen_key": "ingestion_runs",
      "feature_key": "trigger_run",
      "label": "Trigger Ingestion In UI",
      "description": "Grants rights to dispatch ingestion run tasks."
    }
  ]
}
```

- **Business Rules:** Results are cached in the browser's `sessionStorage` with automatic validation against background API checks on layout mount.

SIGNALMDM: DATABASE SCHEMA & REFERENCES MANUAL

This document provides a highly detailed reference of the PostgreSQL database schemas, indexes, structural tenant-isolation strategies, and transactional audit layouts in SignalMDM.

1. Multi-Tenant Isolation Strategy

Logical isolation is enforced directly within the PostgreSQL storage tier.

1. **Strict Scoping Column:** Every core table includes a mandatory column:

```
tenant_id UUID NOT NULL
```

2. **No Direct Deletes:** Foreign key definitions contain `onDelete="RESTRICT"`. This prevents organizational administrative data (e.g. Ingestion runs or Raw records) from being wiped out or orphaned:

```
CONSTRAINT fk_tenant FOREIGN KEY (tenant_id) REFERENCES tenant(tenant_id) ON DELETE RESTRICT
```

3. **Automatic Queries Appends:** Row-level context parameters, decoded from the client's AES-256 JWT by backend middlewares, are bound to the active database session context. All SELECT queries append:

```
WHERE tenant_id = :session_tenant_id
```

This completely isolates query data between corporate organizations.

2. Table Referential Specifications

Here we outline the structural details of key system tables.

2.1 Table: tenant

- **Purpose:** The parent organization identity record. All system schemas reference this root.
- **Columns:**
 - **tenant_id** (UUID, Primary Key): Unique system identifier.
 - **name** (VARCHAR(255), Not Null): Legal company or entity name.
 - **config_json** (JSONB, Nullable): Configuration variables (domain configurations, custom adaptors, mapping parameters).
- **Indexes:**
 - **pk_tenant** (B-Tree on **tenant_id**).
- **Business Meaning:** Establishes organizational boundaries for data residency and billing configurations.

2.2 Table: platform_admin

- **Purpose:** Stores identity records and authentication states for global administrators.
- **Columns:**
 - **admin_id** (UUID, Primary Key): Primary administrator key.
 - **email** (VARCHAR(255), Unique, Not Null): Login email address.
 - **password_hash** (VARCHAR(255), Not Null): password string, salted and hashed using bcrypt.
 - **full_name** (VARCHAR(255), Not Null): Name of the administrator.
 - **status** (VARCHAR(50), Default: 'active'): Current state of the account (**active**, **locked**, **suspended**).

- **failed_login_attempts** (**INTEGER**, Default: **0**): Tracks sequential login failures.
- **locked_until** (**TIMESTAMP**, Nullable): Timestamp indicating when an account lockout expires.
- **Indexes:**
 - **uq_admin_email** (Unique B-Tree on **email**).
- **Query Patterns:**
 - ``sql`

```
SELECT * FROM platform_admin WHERE email = :email;
```

* **Performance Considerations:** The email column is heavily indexed to guarantee sub-millisecond lookups on the initial login pipeline step.

2.3 Table: ``raw_record``

* **Purpose:** The immutable landing zone for ingested records. Files parsed by the data quality loader write directly here.

* **Columns:**

- * ``raw_record_id`` (``UUID``, Primary Key): Primary record key.
- * ``tenant_id`` (``UUID``, Foreign Key, Not Null): Owning tenant ID.
- * ``run_id`` (``UUID``, Foreign Key, Not Null): The ingestion run that generated this record.
- * ``raw_data`` (``JSONB``, Not Null): The raw CSV or JSON data dictionary.
- * ``checksum_md5`` (``VARCHAR(32)``, Not Null): MD5 hash of the row payload, preventing duplicate storage.
- * ``created_at`` (``TIMESTAMP``, Default: ``NOW()``): Creation timestamp.

* **Indexes:**

- * ``idx_raw_checksum`` (B-Tree on ``tenant_id``, ``checksum_md5``).
- * ``idx_raw_run`` (B-Tree on ``run_id``).

* **Query Patterns:**

- * ``sql`
- ```
SELECT * FROM raw_record WHERE tenant_id = :t_id AND checksum_md5 = :md5;
```

- **Performance Considerations:** Features a composite index on (**tenant\_id**, **checksum\_md5**) to check for duplicates before bulk inserting batches of 50,000 records.

## 2.4 Table: `staging_entity`

- **Purpose:** The staging zone where raw records are correlated, verified, and prepared for survivorship evaluation.
- **Columns:**
  - **staging\_id** (**UUID**, Primary Key): Unique record identifier.
  - **tenant\_id** (**UUID**, Foreign Key, Not Null): Owning tenant ID.
  - **raw\_record\_id** (**UUID**, Foreign Key, Unique, Not Null): References the raw record that generated this staging entity.
  - **external\_id** (**VARCHAR(255)**, Not Null): Target source identification key.
  - **state** (**VARCHAR(50)**, Default: `'READY_FOR_MAPPING'`): Current validation status (**READY\_FOR\_MAPPING**, **VALIDATION\_FAILED**, **MAPPED**).
  - **validated\_at** (**TIMESTAMP**, Nullable): Timestamp of the last validation run.
- **Indexes:**
  - **uq\_staging\_raw\_record** (Unique index on **raw\_record\_id**).
  - **idx\_staging\_external** (B-Tree on **tenant\_id**, **external\_id**).
- **Business Meaning:** Represents cleaned transactional data mapped from external models into a unified schema, awaiting matching algorithms.

## 2.5 Table: `audit_log`

- **Purpose:** Immutable append-only audit trail capturing all system mutations.
  - **Columns:**
    - **audit\_id** (**UUID**, Primary Key): Primary record key.
    - **tenant\_id** (**UUID**, Foreign Key, Not Null): References the target tenant.
    - **action** (**VARCHAR(255)**, Not Null): Action performed (e.g. **'CREATE'**, **'UPDATE'**).
    - **table\_name** (**VARCHAR(100)**, Not Null): Table impacted by the change.
    - **record\_id** (**UUID**, Not Null): Primary key of the modified record.
    - **old\_value** (**JSONB**, Nullable): Snapshot of the data dictionary prior to modification.
    - **new\_value** (**JSONB**, Nullable): Snapshot of the data dictionary post-modification.
    - **performed\_by** (**VARCHAR(255)**, Not Null): Username or service identifier that initiated the change.
    - **performed\_at** (**TIMESTAMP**, Default: **NOW()**): Timestamp of the operation.
    - **trace\_id** (**UUID**, Not Null): Trace identifier linking HTTP requests to database changes.
  - **Indexes:**
    - **idx\_audit\_trace** (B-Tree on **trace\_id**).
    - **idx\_audit\_performed** (B-Tree on **performed\_at**).
  - **Business Meaning:** Ensures compliance with security regulations (such as SOC2 or HIPAA) by maintaining a complete, immutable history of master data changes.
- 

### 3. Index Strategy Summary

- **Foreign Key Indexing:** Every foreign key in the schema is backed by a B-Tree index to speed up join operations and avoid full table scans during cascades.
- **Composite Indexing:** Multi-tenant scoping queries are optimized using composite indexes prefixed with the **tenant\_id** column (e.g. **idx\_raw\_checksum** on (**tenant\_id**, **checksum\_md5**)).
- **JSONB Indexing:** The **raw\_data** and metadata **config\_json** columns use generalized inverted index (GIN) models to support nested attribute filtering:

```
CREATE INDEX idx_raw_data_gin ON raw_record USING gin (raw_data);
```

# SIGNALMDM: ENTERPRISE PRODUCTION DEPLOYMENT MANUAL

This document provides a highly detailed specification for configuring, deploying, scaling, and managing the SignalMDM platform in cloud environments.

## 1. Environments Specification

SignalMDM utilizes a standardized environment topology to guarantee smooth delivery paths:

| Profile     | Purpose                            | Deployment Strategy                       | Isolation Type                    |
|-------------|------------------------------------|-------------------------------------------|-----------------------------------|
| Development | Sandbox / Local engineering        | Local host execution / Single-node Docker | Logical                           |
| QA          | Automated testing & validation     | Docker Compose on virtualized host        | Logical                           |
| UAT         | Client acceptance & staging checks | Multi-node Kubernetes (EKS / GKE)         | Shared Database, Dedicated Schema |
| Production  | Public master systems              | High Availability Kubernetes Cluster      | Dedicated Database Instance       |

## 2. Infrastructure Requirements

### 2.1 Compute Targets

- Web Frontend Pods:** 2x Replica nodes, 0.5 CPU Cores, 512MB RAM per pod.
- Backend API Pods:** 3x Replica nodes, 1.0 CPU Cores, 1.5GB RAM per pod.
- Celery Workers Pods:** 2x Replica nodes, 2.0 CPU Cores, 4.0GB RAM per pod (intensive Data Quality sanitization runs).

### 2.2 Storage & Network

- PostgreSQL Engine:** Managed instance (e.g. AWS RDS PostgreSQL 15) with multi-AZ replication. Minimum 4 vCPUs, 16GB RAM, 100GB provisioned IOPS SSD storage.
- Caching (Redis):** Elasticache Redis instance, minimum 2 Cores, 4GB RAM.
- Network:** Private subnet boundaries isolating Database and Caching layers, with access restricted to the Kubernetes cluster security group.

## 3. Container Configurations & Orchestration

### 3.1 Backend Dockerfile (MDM\_Backend/Dockerfile)

```
FROM python:3.12-slim as builder

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
 build-essential \
 libpq-dev \
```

```
&& rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir --user -r requirements.txt

FROM python:3.12-slim as runner

WORKDIR /app

RUN apt-get update && apt-get install -y --no-install-recommends \
 libpq5 \
 && rm -rf /var/lib/apt/lists/*

COPY --from=builder /root/.local /root/.local
COPY . .

ENV PATH=/root/.local/bin:$PATH
ENV APP_ENV=production

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000", "--no-server-header"]
```

---

## 3.2 System Multi-Container Orchestration (docker-compose.yml)

For QA and staging environments:

```
version: '3.8'

services:
 database:
 image: postgres:15-alpine
 container_name: mdm_database
 environment:
 POSTGRES_DB: SignalMDM
 POSTGRES_USER: postgres
 POSTGRES_PASSWORD: production_password
 ports:
 - "5432:5432"
 volumes:
 - pgdata:/var/lib/postgresql/data
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U postgres"]
 interval: 5s
 timeout: 5s
 retries: 5

 redis:
 image: redis:7-alpine
 container_name: mdm_redis
 ports:
 - "6379:6379"
 healthcheck:
 test: ["CMD", "redis-cli", "ping"]
 interval: 5s
 timeout: 5s
 retries: 5

 api:
```

```
build:
 context: ./MDM_Backend
 dockerfile: Dockerfile
container_name: mdm_api
environment:
 - DATABASE_URL=postgresql://postgres:production_password@database:5432/SignalMDM
 - REDIS_URL=redis://redis:6379/0
 - APP_ENV=production
 - JWT_SECRET=production_jwt_secret
 - TOKEN_ENCRYPTION_KEY=a3f1b2c4d5e6f7a8b9c0d1e2f3a4b5c6d7e8f9a0b1c2d3e4f5a6b7c8d9e0f1a2
ports:
 - "8000:8000"
depends_on:
 database:
 condition: service_healthy
 redis:
 condition: service_healthy

worker:
 build:
 context: ./MDM_Backend
 dockerfile: Dockerfile
 container_name: mdm_worker
 command: celery -A signalmdm.workers.celery_app worker --loglevel=info
 environment:
 - DATABASE_URL=postgresql://postgres:production_password@database:5432/SignalMDM
 - REDIS_URL=redis://redis:6379/0
 - APP_ENV=production
 depends_on:
 redis:
 condition: service_healthy

volumes:
 pgdata:
```

---

### 3.3 Kubernetes Deployment Resource Specs (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
 name: mdm-api-deployment
 namespace: mdm-production
 labels:
 app: mdm-api
spec:
 replicas: 3
 selector:
 matchLabels:
 app: mdm-api
 template:
 metadata:
 labels:
 app: mdm-api
 spec:
 containers:
 - name: mdm-api-container
 image: mdm-registry.corp/signalmdm-backend:1.0.0
 ports:
 - containerPort: 8000
```

```
envFrom:
- secretRef:
 name: mdm-production-secrets
resources:
 limits:
 cpu: "1.5"
 memory: 2Gi
 requests:
 cpu: "500m"
 memory: 512Mi
livenessProbe:
 httpGet:
 path: /health
 port: 8000
 initialDelaySeconds: 15
 periodSeconds: 10
readinessProbe:
 httpGet:
 path: /health
 port: 8000
 initialDelaySeconds: 10
 periodSeconds: 5
```

## 4. Disaster Recovery & Operational Runbooks

### 4.1 Automated Database Backup Script (backup.sh)

```
#!/bin/bash
set -e

BACKUP_DIR="/mnt/backups/postgres"
TIMESTAMP=$(date +"%Y%m%d_%H%M%S")
DATABASE_NAME="SignalMDM"
PGPASSWORD="production_password"
BACKUP_FILE="${BACKUP_DIR}/${DATABASE_NAME}_backup_${TIMESTAMP}.sql"

mkdir -p "$BACKUP_DIR"

echo "Starting automated database backup for database ${DATABASE_NAME}..."
PGPASSWORD="$PGPASSWORD" pg_dump -h localhost -U postgres -d "$DATABASE_NAME" -F p -f "$BACKUP_FILE"

echo "Compressing backup file..."
gzip "$BACKUP_FILE"

echo "Exporting compressed backup file to secure AWS S3 bucket..."
aws s3 cp "${BACKUP_FILE}.gz" "s3://corp-mdm-backups/database/${DATABASE_NAME}_backup_${TIMESTAMP}.sql.gz"

echo "Database backup and export completed successfully."
```

### 4.2 Rollback Procedure Runbook

If an active deployment fails liveness or integration checks:

1. **Halt CI/CD pipeline progression:** Trigger pipeline cancellation.
2. **Locate the last stable container tag:** E.g. **v1.0.0-RC1**.
3. **Execute Kubernetes Rollback command:**

```
kubectl rollout undo deployment/mdm-api-deployment -n mdm-production
```

#### 4. Confirm rollback status:

```
kubectl rollout status deployment/mdm-api-deployment -n mdm-production
```

#### 5. Verify Pod state and health endpoints:

```
kubectl get pods -n mdm-production
curl -I http://mdm-production.corp/api/v1/health
```

---

## 5. Troubleshooting & Diagnostics

### 5.1 Issue: Redis Out of Memory (OOM)

- **Symptom:** API requests hang or return **500 Internal Server Error**, Celery workers stop accepting raw ingestion tasks.
- **Diagnostic:** Check Redis memory usage statistics:

```
redis-cli -h localhost info memory
```

If **used\_memory\_human** matches the allocated limits, Redis is out of memory.

- **Resolution:**
  1. Clear expired token revocation keys from the database:

```
redis-cli keys "revoked:*" | xargs redis-cli del
```

2. Check the size of the Celery broker queues:

```
redis-cli llen celery
```

3. If queues are persistently overloaded, scale background worker pods:

```
kubectl scale deployment/mdm-worker-deployment --replicas=5 -n mdm-production
```

# SIGNALMDM: ENTERPRISE TECHNICAL REFERENCE GUIDE

This document provides a deep-dive technical reference on the frameworks, libraries, design architectures, and dependencies powering the SignalMDM enterprise application.

## 1. Core Technical Components

Here we detail the purpose, context, and integration flows of our core technology decisions.

### 1.1 FastAPI

- **Purpose:** The primary asynchronous web API framework.
- **Where Used:** Backend routing controller plane (`MDM_Backend/main.py` and routers).
- **Files Utilizing:** `main.py`, `routers/auth_router.py`, `routers/ingestion_router.py`.
- **Why It Is Used:** Standardizes API development by leveraging Pydantic models for request/response serialization. Native support for `async/await` ensures efficient concurrent request handling under high loads.
- **Interaction Model:** Receives incoming JSON payloads, validates them using Pydantic, validates access scopes using dependency injection guards in `auth.py`, and delegates transaction handling to services.

### 1.2 Celery

- **Purpose:** Asynchronous distributed task execution queue.
- **Where Used:** Background processing of large ingestion runs (`MDM_Backend/signalmdm/workers`).
- **Files Utilizing:** `celery_app.py`, `raw_worker.py`, `staging_worker.py`.
- **Why It Is Used:** Decouples intensive data cleansing, deduplication, and staging operations from the HTTP request-response cycle, preventing timeouts and ensuring service stability.
- **Interaction Model:** The FastAPI ingestion router queues jobs via `celery.send_task`. Celery workers consume tasks from Redis, execute SQL updates, and update the execution status in PostgreSQL.

### 1.3 React & TypeScript

- **Purpose:** Client application engine.
- **Where Used:** Frontend user interface layout (`MDM_Frontend/src`).
- **Files Utilizing:** `main.tsx`, `App.tsx`, `pages/UploadData.tsx`, `context/AuthContext.tsx`.
- **Why It Is Used:** TypeScript ensures type-safe component state management, preventing runtime UI exceptions. Vite provides extremely fast build compilation times.
- **Interaction Model:** Standardizes API integration via Axios client configurations, manages sessions in global React Context wrappers, and enforces route protection using state properties.

### 1.4 PostgreSQL

- **Purpose:** Authoritative enterprise relational database.
- **Where Used:** Persistent data storage tier (`MDM_DataLayer/SignalMDM.sql`).
- **Why It Is Used:** Excellent JSONB performance, transaction compliance, referential integrity safeguards (`ondelete="RESTRICT"`), and mature composite indexing options.
- **Interaction Model:** Coordinates relational structures across tenants, administrators, ingestion runs, raw immutable records, and staging assets.

## 1.5 Redis

- **Purpose:** Low-latency caching engine, rate limiter, and task message broker.
- **Where Used:** Backend middleware and task queues.
- **Files Utilizing:** `core/redis_client.py`, `middleware/rate_limit.py`, `services/auth_service.py`.
- **Why It Is Used:** Delivers single-digit millisecond responses for token revocation checks, sliding-window rate limiting, and OTP caching.
- **Interaction Model:** Stores revoked JWT signatures, salted/hashed OTP codes (with 10-minute TTLs), and tracks login attempts to prevent brute-force attacks.

## 2. Major Dependency Catalog

Below is a detailed inventory of the primary third-party dependencies used in SignalMDM.

| Dependency Name | Inspected Version | Purpose                                          | Usage Files                                                         | Architectural Benefits                                                   | Alternatives               |
|-----------------|-------------------|--------------------------------------------------|---------------------------------------------------------------------|--------------------------------------------------------------------------|----------------------------|
| FastAPI         | 0.136.1           | REST API orchestration and dependency injection. | <code>main.py</code> , <code>routers/*.py</code>                    | Asynchronous speed, automatic OpenAPI generation, clean type validation. | Django REST, Flask         |
| Uvicorn         | 0.42.0            | High-speed ASGI server execution.                | Startup scripts                                                     | Ultra-low overhead, handles high-concurrency connections.                | Hypercorn, Gunicorn        |
| SQLAlchemy      | 2.0.40            | Transaction-safe database ORM interface.         | <code>database.py</code> , <code>models/*.py</code>                 | Unified query generation, connection pooling, complex mapping controls.  | TortoiseORM, Peewee        |
| Pydantic        | 2.11.7            | Standard data parsing and serialization.         | <code>schemas/*.py</code> , <code>core/config.py</code>             | Fast data validation, clear runtime typing errors.                       | Marshmallow, Cerberus      |
| Celery          | 5.6.3             | Background asynchronous task distribution.       | <code>workers/celery_app.py</code>                                  | Proven task queues, automatic retries with exponential backoffs.         | RQ (Redis Queue), Dramatiq |
| Redis           | 6.2.0             | Cache memory storage and messaging client.       | <code>core/redis_client.py</code>                                   | Fast keyspace operations, sliding-window rate-limiting.                  | Memcached, RabbitMQ        |
| Python-Jose     | 3.5.0             | JWT token signature coding and decoding.         | <code>middleware/token_utils.py</code>                              | Robust support for HS256 algorithm pinning, secure claim checks.         | PyJWT                      |
| Cryptography    | 46.0.5            | Standard AES-256-CBC token encryption wrapper.   | <code>middleware/token_utils.py</code>                              | Strong client-server session envelope security.                          | PyCryptodome               |
| Bcrypt          | 5.0.0             | Adaptive hashing of sensitive values.            | <code>services/auth_service.py</code>                               | High computational complexity prevents database dump cracking.           | Argon2                     |
| PyOTP           | 2.9.0             | Encoded OTP code calculation algorithms.         | <code>services/auth_service.py</code>                               | Seamless integration of standard MFA workflows.                          | Custom OTP logic           |
| Crypto-JS       | 4.2.0             | Client-side AES-256 encryption engine.           | <code>utils/crypto.ts</code> , <code>context/AuthContext.tsx</code> | Secure client-side payload obfuscation before transit.                   | Web Crypto API             |

### 3. Core Architectural Patterns

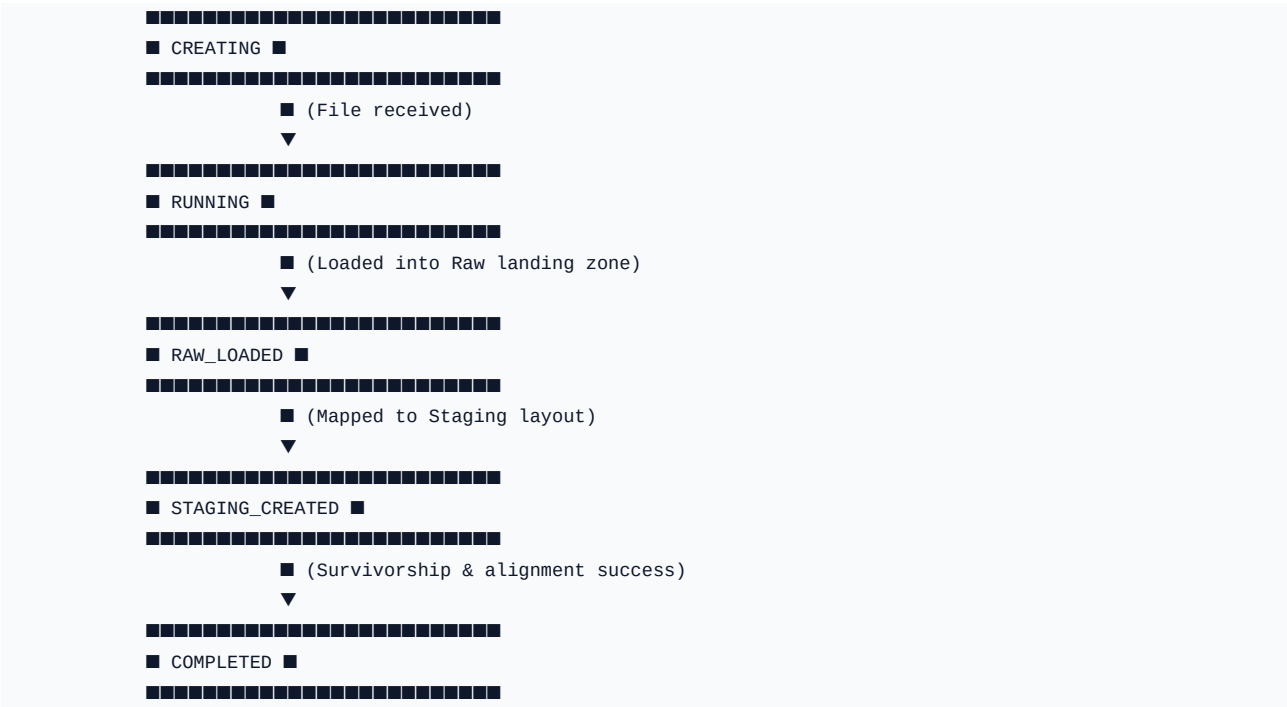
#### 3.1 Encrypted Token Security Pattern

The platform implements an encrypted JWT payload pattern to protect credentials in transit:



#### 3.2 Ingestion Engine State Machine

The async ingestion pipeline implements a transaction-safe state machine to track data processing:



If any stage fails (e.g. data quality anomalies or invalid CSV formats), the state transitions to **FAILED**, and the specific error is logged in the **IngestionRun** audit record.